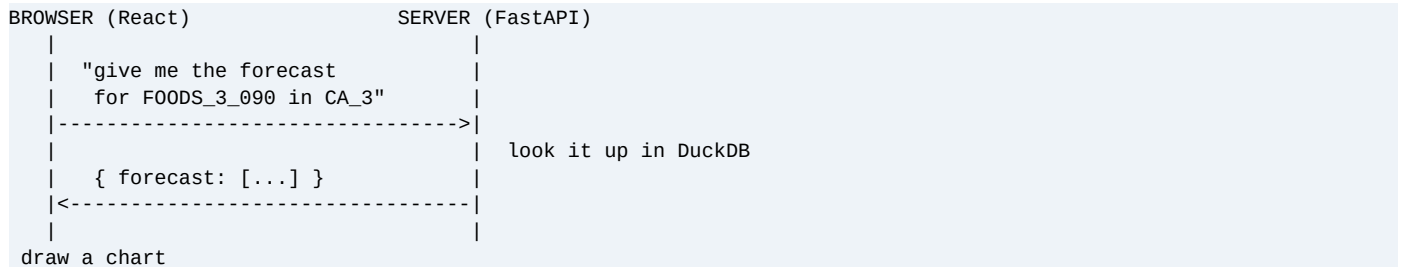


Backend Developer — Detailed Guide

Retail Demand Forecasting (Walmart M5, Hierarchical) · Team Study Material

1. What a backend is, using our project as the example

The **frontend** is what you see in the browser — buttons, charts, pages. The **backend** is a program running elsewhere that holds the data and answers questions about it. The browser cannot open our 125 MB database directly, so it asks the backend instead.



API — Application Programming Interface. A defined way for two programs to talk to each other. Not a user interface: no buttons, no colours. Just requests and structured answers. In our project the React frontend uses the FastAPI backend's API to ask for forecast information. **HTTP — HyperText Transfer Protocol.** The language of those requests. A request has a **method** (GET = fetch something, POST = send something), a **path** (/api/v1/health), and gets back a **status code** (200 = fine, 404 = not found, 503 = service unavailable) plus a body. **JSON — JavaScript Object Notation.** The format of the body: text made of {"key": value} pairs and [lists]. Both Python and JavaScript read it natively, which is why it is the standard.

Why our project needs a backend

Three reasons, all real:

- **The data is too big for a browser.** The product data layer is ~125 MB, and the history alone is 59.2 million rows. The browser gets only the slice it asks for.
- **The frozen model lives on the server.** Model files and forecasts stay server-side, read-only. The browser never touches them.
- **The Gemini API key must never reach the browser.** This is a security requirement, not a preference. Only the backend holds it. (See the GenAI topic.)

2. What FastAPI is

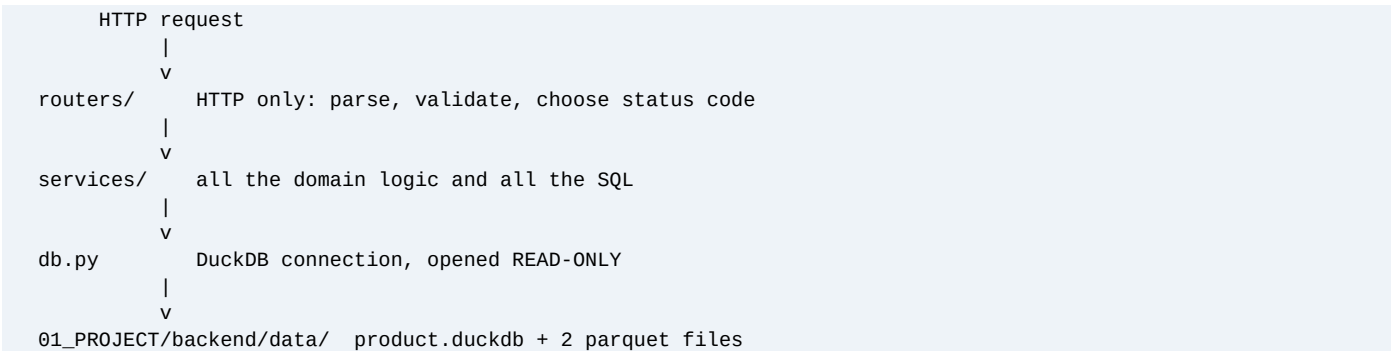
FastAPI is a Python framework for building APIs. Two things make it a good fit here:

- **Automatic validation.** You declare what a parameter should look like and FastAPI rejects bad requests before your code runs.
- **Automatic documentation.** It generates an interactive page at `/docs` and a machine-readable schema at `/openapi.json` from the code itself, so the documentation cannot drift from reality.

Endpoint / route. One address the API answers on — e.g. `GET /api/v1/series/{store_id}/{item_id}/forecast`. The `{curly}` parts are variables filled in by the caller. **Uvicorn.** The web server that actually runs our FastAPI app and listens on a port. FastAPI defines *what* to answer; uvicorn handles the network.

VERIFIED: our API exposes 34 endpoints and has 149 passing tests.

3. Our architecture — three layers



Why separate them? A router should not know SQL, and a service should not know about HTTP status codes. When something breaks you know which layer to open. It also means the services can be tested without pretending to be a web server.

The actual modules

Path	Responsibility
app/main.py	creates the app, registers routers, error handlers, CORS
app/config.py	every setting, environment-overridable
app/db.py	read-only DuckDB access, identifier whitelisting
app/cache.py	small in-process cache with a time limit
app/errors.py	the error types (NotFound, BadRequest, ServiceUnavailable, ...)
app/schemas/models.py	response shapes
app/routers/	health, meta, hierarchy, series, accuracy, insights, inference, genai
app/services/	one service per router, plus genai_context, calendar
app/worker/runner.py	background job runner for the long inference job

4. The endpoints

All 34, grouped by router:

Group	Endpoints
Health	GET /health, GET /ready
Meta	GET /meta/model, /meta/capabilities, /meta/provenance
Hierarchy	GET /hierarchy/levels, /nodes, /search, /aggregate
Series	GET /series, /series/{store}/{item}, /history, /forecast
Accuracy	GET /accuracy/windows, /levels, /horizon, /regimes, /occurrence, /volume-tiers, /members, /error-bands, /backtest, /backtest/{store}/{item}
Insights	GET /insights/summary, /top-movers, /planning/{store}/{item}
Inference	GET /inference/status, /jobs, /jobs/{id}, POST /inference/verify
GenAI	GET /genai/status, /genai/suggestions, POST /genai/ask, /genai/context-preview

All live under the prefix /api/v1.

Why /api/v1? Versioning. If we ever needed to change a response shape we could add /api/v2 without breaking anything already using v1.

Three endpoints in detail

GET /api/v1/health **and** GET /api/v1/ready

/health answers instantly and touches no data — it only says the process is alive. /ready opens DuckDB and counts rows in the expected tables, returning ready: true/false plus degraded if a sidecar file is missing.

Liveness vs readiness — a real distinction. Liveness = "the process is running". Readiness = "it can do its job". A container can be alive but not ready for several seconds after start, or if its data volume is missing. Our container healthcheck uses /ready, deliberately, because a process that is up but cannot answer questions is not useful.

GET /api/v1/series/{store_id}/{item_id}/forecast

WHAT DOES IT DO? Returns the frozen 28-day forecast for one product in one store. **FRONTEND SENDS:** store and item in the path, e.g. /series/CA_3/FOODS_3_090/forecast. **BACKEND DOES:** validates the identifiers, looks up the forecast rows in DuckDB, attaches the empirical error band for that product's demand regime and horizon. **RETURNS:** 28 daily values, a 28-day total, the planning range, and a band_basis field stating in words that the range is observed past error and not a model-produced interval.

Verified example: CA_3 / FOODS_3_090 → total_28d = 3331.3681.

POST /api/v1/genai/ask

WHAT DOES IT DO? Answers a natural-language question about the forecast. **FRONTEND SENDS:** JSON — {"question": "...", "store_id": "...", "item_id": "...". **BACKEND DOES:** checks local policy first; if the question is an attack it refuses immediately with no AI call. Otherwise it gathers verified numbers from the same services the rest of the API uses, sends that small context to Gemini, and checks every number in the reply against the context. **RETURNS:** the answer plus provenance — grounded, ungrounded_numbers, intent, elapsed_ms, refused.

GET vs POST. GET fetches something and should change nothing. POST sends a body. ask is a POST because the question is a body, not a URL.

Request to response, end to end

```
1 Browser      GET /api/v1/series/CA_3/FOODS_3_090/forecast
2 nginx/proxy  forwards /api/ to the backend
3 main.py      matches the route
4 routers/series validates store_id and item_id
5 services/series builds and runs the SQL
6 db.py        DuckDB answers (read-only)
7 cache.py     stores the result briefly
8 schemas     response is shaped and type-checked
9 Browser      200 + JSON -> chart is drawn
```

If step 4 fails → 400 Bad Request. If the product does not exist → 404 Not Found. If the database is missing → 503 Service Unavailable, with a reason.

5. Data access, validation and errors

The database is opened read-only

`db.py` opens DuckDB in read-only mode. The API physically cannot write to the product data, and in the container the file is mounted `:ro` as well. Two independent layers saying the same thing.

Identifier whitelisting

Some queries need a table or level name chosen by the caller (for example `level=store`). Those cannot be passed straight into SQL.

SQL injection. An attack where a caller sends something like `store'; DROP TABLE ...` and it gets executed as code.

We do not sanitise — we **whitelist**. `db.py` holds a `SAFE_LEVELS` set, and anything not in that exact list is rejected before any SQL is built. A test sends injection strings and asserts they are rejected.

Validation

FastAPI validates types and ranges from the declarations. Beyond that we cap list sizes (`max_list_limit = 500`) and history length (`max_history_days`), so one request cannot ask for everything at once.

Errors

`errors.py` defines the error types and every one produces the same JSON shape:

```
{ "error": "service_unavailable",
  "message": "...",
  "context": { "remedy": "..." } }
```

Consistency matters: the frontend has one error path, not eight. Where we can suggest a fix, `context.remedy` carries it.

CORS

CORS — Cross-Origin Resource Sharing. A browser rule: a page served from one origin may not call a different origin unless that server allows it.

We **configure** CORS (`NPN_CORS_ORIGINS`) but the production topology does not exercise it: nginx serves the app and proxies `/api/` to the backend, so the browser sees a single origin. CORS matters only during development, when the Vite dev server runs on port 5173 and the API on 8000.

Configuration — no hard-coded paths

Every setting lives in `app/config.py` and is overridable by an environment variable with the `NPN_` prefix.

Environment variable. A value supplied to a program by its surroundings rather than written in the code. It lets the same code run on a laptop and in a container with different paths.

Variable	Purpose
<code>NPN_DATA_DIR</code>	where the product data lives
<code>NPN_PROJECT_ROOT</code>	where the research tree is (optional)
<code>NPN_ENABLE_INFERENCE</code>	turn live verification on/off
<code>NPN_CORS_ORIGINS</code>	allowed browser origins

Variable	Purpose
GEMINI_API_KEY	the AI key — runtime only, never in code

A test asserts there is no hard-coded Windows path anywhere in app/, and another proves the API serves correctly with the research tree entirely absent.

6. How the backend meets the model and Gemini

Model integration

Two distinct paths, and the difference matters:

Path 1 — serving (always). The forecast was computed once in research and stored. The API reads it from DuckDB. **No model is loaded**, nothing is computed. This is why the API starts in well under a second and needs no ML libraries.

Path 2 — live verification (optional). `POST /api/v1/inference/verify` actually reloads both frozen LightGBM models, rebuilds features, re-runs the blend and compares against the shipped forecast. It takes ~48 s, so it runs as a **background job**: the request returns a job id immediately and the frontend polls `/inference/jobs/{id}`.

VERIFIED: `max_abs_diff = 0.0000e+00` — *the live model reproduces the shipped forecast exactly.*

A deliberate architectural decision: the API does **not** import the research pipeline at startup. Importing `pipeline.config` creates directories as a side effect — a write to the protected tree. It is imported lazily inside the inference service only, so a container built without LightGBM still serves every other endpoint and reports why inference is unavailable.

Connecting the frontend to GenAI

```
React --POST /api/v1/genai/ask--> FastAPI
                                   |
                                   local policy check (may refuse here)
                                   |
                                   gather verified numbers from our own services
                                   |
                                   Gemini API
                                   |
                                   check every number against the context
                                   |
React <----- answer + provenance --+
```

The browser never contacts Gemini. It has no key, no endpoint and no SDK. The key is held server-side as a masked secret and is never placed in a prompt, a response or a log.

7. Testing

VERIFIED: 149 backend tests pass.

File	Tests	What it protects
test_genai.py	71	guardrails, key never leaks, grounding
test_hierarchy.py	16	coherence, level accuracy, injection rejection
test_accuracy.py	12	published metrics, no window without ground truth
test_integrity.py	10	the freeze guard — model hashes, forecast row-for-row
test_series.py	10	history ordering, band bracketing and wording
test_insights.py	9	totals agree across endpoints
test_deployment.py	8	env config, no Windows paths, portability
test_meta.py	5	model card values, capability matrix
test_health.py	4	liveness, readiness, request-id contract
test_inference.py	4	availability, refusals, job lifecycle

The tests worth mentioning aloud are the ones asserting **scientific** invariants, not HTTP codes: that children sum to their parent, that the served forecast still matches the frozen file, that no accuracy is ever returned for a window with no ground truth.

8. Implementation decisions worth defending

A separate product data layer. Early on the API queried the 287 MB research parquet directly. That was replaced because DuckDB bakes **absolute paths** into views over external files — the database literally contained a Windows path, which is fatal in a Linux container. The rebuilt sidecar is **31.7 MB instead of 287 MB** and answers a single-series query in **8–13 ms instead of ~320 ms**.

Single worker. DuckDB opens read-only per process, and each extra worker would add a full model and panel copy during inference. Scale with replicas, not in-process workers.

Read-only everything. Read-only DuckDB, :ro mounts, non-root container user. Three independent mechanisms preventing the same mistake.

9. Common misunderstandings

"The backend runs the model on every request." No. It reads a precomputed forecast. The model only runs on the optional verification endpoint.

"The API needs the research data." No — verified by a test that runs it with an empty project root.

"CORS is what secures the API." No. CORS is a browser rule about who may call. It is not authentication.

"/health and /ready are the same." No. Health = the process is up. Ready = it can serve. The container healthcheck uses /ready.

"The frontend talks to Gemini." Never. It talks only to our backend.

10. How to present this

"I built the backend — the part that sits between the browser and our data.

It's FastAPI, which is a Python framework for building APIs. An API is just a defined way for two programs to talk: the React app sends an HTTP request like 'give me the forecast for this product in this store', and gets back JSON.

We have 34 endpoints in three layers — routers handle HTTP, services hold the logic and the SQL, and a database module that opens DuckDB read-only. Keeping them separate means a router never contains SQL and a service never knows about status codes.

The important design decision is that the backend doesn't run the model on every request. The forecast was computed once and stored, so a request is just a database lookup — 8 to 13 milliseconds. There's a separate optional endpoint that actually reloads the frozen model and re-runs everything to prove it still reproduces the shipped forecast exactly.

Security-wise: the database is opened read-only, the model files are mounted read-only, the container runs as a non-root user, and level names are whitelisted rather than sanitised so SQL injection isn't possible. And the Gemini key lives only on the server — the browser never sees it.

There are 149 tests, and the ones I'd point at aren't the HTTP checks — they're the ones asserting that children sum to their parent, and that the forecast we serve still matches the frozen file."

11. Likely judge questions

Beginner

Q: What is an API? A: A defined way for two programs to talk. Our React frontend sends HTTP requests to the FastAPI backend and gets JSON back.

Q: What does the backend actually do? A: Holds the data, answers questions about the forecast and accuracy, and talks to Gemini on the frontend's behalf so the key stays server-side.

Q: How many endpoints? A: 34, all under `/api/v1`, with interactive docs at `/docs`.

Technical

Q: Why FastAPI? A: Automatic request validation and automatically generated OpenAPI docs, so the documentation can't drift from the code.

Q: How do you stop SQL injection? A: We whitelist rather than sanitise — level and table names must appear in a fixed `SAFE_LEVELS` set or the request is rejected before any SQL is built. There's a test that sends injection strings.

Q: Why `/ready` as the healthcheck instead of `/health`? A: Health only proves the process is alive. Ready proves the data layer opens and is queryable. A container with a missing data volume is alive but useless.

Q: Does the API need the research tree? A: No, and we prove it — a test launches the app with an empty project root and verifies forecasts, hierarchy and accuracy all still serve.

Q: Why not more uvicorn workers? A: DuckDB opens read-only per process, and each worker adds a full model and panel copy during inference. We scale with replicas instead.

Q: How is the Gemini key protected? A: It's an environment variable held as a masked secret, never in code, never in a prompt, never in a response, never in the frontend bundle. Tests check all four GenAI endpoints against the configured key.

12. Glossary

Term	Meaning
API	Application Programming Interface — how two programs talk
HTTP	The request/response protocol of the web
GET / POST	Fetch something / send a body
Status code	200 fine, 400 bad request, 404 not found, 503 unavailable
JSON	The text data format used for request and response bodies
Endpoint / route	One address the API answers on
FastAPI	The Python framework we use
Uvicorn	The server process that runs the app
Router / service	HTTP layer / logic-and-SQL layer
DuckDB	The in-process analytics database
CORS	Browser rule about calling a different origin
Environment variable	A setting supplied from outside the code
Liveness / readiness	Process is up / process can serve
SQL injection	Sending SQL as data so it gets executed
Whitelist	Allow only known-good values; reject everything else